

# Rigorous Methodological Audit & De-Biasing Framework for Respiratory Audio Diagnostics

## Module 1: Acoustic Confounding & Adversarial De-Biasing Architecture

Crowdsourced respiratory audio datasets exhibit wide device and environmental variability. For example, the Coswara data collection explicitly records each participant's **device type** and **ambient environment** along with demographics [4†L630-L639]. Its manual quality audit found ~10% of recordings to have significant noise ("poor" quality) [4†L724-L733]. Similarly, the COUGHVID corpus is global and crowdsourced; it includes a 51-column metadata file with fields such as an expert "quality" label (good/ok/poor) and estimated SNR for each recording [14†L848-L856] [14†L904-L913]. These metadata fields make clear that confounding artifacts – smartphone microphone frequency responses, variable sampling rates, room reverberation, background noise and compression codecs – vary systematically across samples. For instance, COUGHVID audio was captured via web browsers in compressed Opus (.webm) format (metadata shows "encoder: Chrome" in example data [14†L848-L856]), whereas Coswara samples come from diverse smartphones at different sample rates. Even speaker accent and language (Coswara is mostly Indian languages; COUGHVID spans many regions) can correlate with COVID labels, risking spurious classification signals.

To **de-bias** against such confounders, we adopt an adversarial multi-task learning framework with a gradient reversal layer (GRL) [19†L143-L151] [20†L7-L15]. Let  $x_i$  be a multi-modal audio input (cough, breath, speech triplet),  $y_i$  the true respiratory condition (e.g. COVID vs healthy), and  $c_i$  a confounder label (e.g. device type, noise category, or demographic group). A shared encoder  $f_\theta(x)$  produces latent features  $z = f_\theta(x)$ . Two classifier heads operate on  $z$ : a **respiratory classifier**  $g_\phi(z)$  (predicting  $y$ ) and a **confounder classifier**  $d_\psi(z)$  (predicting  $c$ ). We define the clinical loss and confound loss as

$$L_{\text{clinical}}(\theta, \phi) = \frac{1}{N} \sum_{i=1}^N \ell(g_\phi(f_\theta(x_i)), y_i), \quad L_{\text{conf}}(\theta, \psi) = \frac{1}{N} \sum_{i=1}^N \ell(d_\psi(f_\theta(x_i)), c_i),$$

where  $\ell$  is cross-entropy. The **total objective** trains  $\theta, \phi, \psi$  with opposing goals:

$$L_{\text{total}} = L_{\text{clinical}}(\theta, \phi) - \lambda L_{\text{conf}}(\theta, \psi).$$

Here the encoder parameters  $\theta$  are optimized to *minimize*  $L_{\text{clinical}}$  (improve disease prediction) while *maximizing*  $L_{\text{conf}}$  (confounder classifier does poorly), whereas the confounder head  $\psi$  is trained to minimize  $L_{\text{conf}}$ . In practice we implement this by inserting a Gradient Reversal Layer before the confound head: during backpropagation GRL multiplies the gradient by  $-\lambda$  [20†L7-L15]. Thus the encoder is forced to produce **domain-invariant features** that confuse the confounder classifier [19†L143-L151] [20†L7-L15]. We tune the hyperparameter  $\lambda$  to balance the tradeoff.

**Key points:** Device and environment labels from Coswara/COUGHVID define the confounder classes (e.g. phone model categories, noise quality bins). The encoder is adversarially trained via GRL to maximize the entropy of these confounder predictions. Mathematically, we alternate gradient steps on

$(\theta, \phi, \psi)$  so that  $\theta$  is updated by  $\nabla_{\theta}(L_{\text{clinical}} - \lambda L_{\text{conf}})$  and  $\psi$  by  $\nabla_{\psi} L_{\text{conf}}$ . This adversarial multi-task objective ensures the learned features capture respiratory pathology while being uninformative about recording artifacts or demographics [19†L143-L151] .

## Module 2: Self-Supervised Audio Features & Transformer-Based Fusion

We eschew handcrafted MFCCs or simple spectrogram CNNs and instead leverage **frozen self-supervised audio encoders**. For each modality (Cough, Breath, Speech), we pass the raw waveform through a pretrained transformer-based backbone (e.g. Wav2Vec 2.0 or HuBERT) to obtain a high-dimensional latent embedding sequence. For example, Wav2Vec 2.0 uses a convolutional encoder to map raw audio  $X$  to a sequence of latent vectors  $z_1, \dots, z_T$ , which are then input to a Transformer for contextualization [50†L123-L131] . In our implementation, we load such a model from HuggingFace (e.g. `facebook/wav2vec2-base`), pass each audio channel through it in evaluation mode (no gradient), and extract the last hidden layer representations  $H_c, H_b, H_s$  for cough, breath, speech respectively. Each  $H_m$  is a 2D tensor of shape (time steps  $\times$  feature dim).

To fuse modalities, we replace naive averaging with **cross-modal attention**. We apply Transformer-style multi-head cross-attention layers so that each modality can attend to the others. Concretely, we linearly project each modality’s features into query, key, and value spaces: e.g.

$$Q_c = H_c W^Q, \quad K_b = H_b W^K, \quad V_b = H_b W^V, \quad K_s = H_s W^K, \quad V_s = H_s W^V.$$

Then the cross-attention output for cough as “query” and (breath, speech) as “sources” is

$$A_{c \rightarrow (b,s)} = \text{softmax}\left(\frac{Q_c [K_b; K_s]^T}{\sqrt{d_k}}\right) [V_b; V_s],$$

where  $\sqrt{d_k}$  is a scaling factor. Similarly we compute  $A_{b \rightarrow (c,s)}$  and  $A_{s \rightarrow (c,b)}$ . In multi-head form, each head computes such a  $\text{Softmax}(QK^T)V$  interaction in parallel and the results are concatenated

[46†L88-L94] . This lets the model learn non-linear acoustic dependencies across modalities (e.g. how cough patterns correlate with breathing/speech features). In effect, cross-attention builds a rich joint representation: a Transformer fusion block is inserted after the SSL backbones, with residual connections to ensure stable gradients.

**Technical detail:** Using the notation above, one can implement in PyTorch with

`torch.nn.MultiheadAttention(embed_dim, num_heads)`, feeding one modality’s queries and another’s keys/values. For example:

```
attn_c_b = nn.MultiheadAttention(embed_dim, heads)
A_c_b, _ = attn_c_b(Q_c, K_b, V_b)  # cough attends to breath
...
```

These outputs are merged (e.g. summed or concatenated) and fed through normalization and feed-forward layers, as in standard Transformer encoders. Importantly, the Query comes from one modality and the Key/Value from the others, so that the fusion explicitly learns cross-modal alignments [46†L88-L94] . This is far more expressive than simple late fusion; mathematically it ensures that the fused feature  $H_{\text{fused}}$  integrates context across all three channels via the attention maps.

## Module 3: Cross-Dataset Covariate-Shift Validation

We define a *rigid cross-dataset evaluation protocol* to quantify out-of-distribution (OOD) performance drops. Concretely, we train the full model on one dataset and test it on the other **without fine-tuning**. For instance, train on Coswara and compute predictions on a held-out COUGHVID test set, and vice versa. We measure standard metrics (accuracy, AUC, sensitivity, specificity, etc.) for in-dataset evaluation (e.g. train on Coswara / test on Coswara) and for cross-dataset evaluation (train Coswara / test COUGHVID). The **absolute generalization drop** is defined as the difference between in-dataset and cross-dataset scores. For example, if  $M_{\text{intra}} = 0.90$  and  $M_{\text{cross}} = 0.65$ , the drop is  $\Delta = 0.25$ . (We report  $\Delta$  for each metric.) We also report relative drop  $\Delta/M_{\text{intra}}$ . All results should include confidence intervals or significance testing (e.g. by bootstrapping) to ensure rigor.

This evaluation is **mandatory** (not optional). Recent work confirms that even high-performing cough classifiers can degrade significantly across datasets [33†L99-L105]. For example, Islam *et al.* (2026) evaluated COVID cough models across five different datasets and found “demographic and geographic differences in the cough sounds” that challenge generalization [33†L99-L105]. Our protocol explicitly mirrors this: it quantifies how performance collapses when encountering “completely unmapped” recording conditions. By treating Coswara and COUGHVID as separate domains, we force the model to reveal its true robustness. All numeric results will be tabulated (e.g. with a matrix showing train/test pairs and metric differences) and visualized (e.g. bar plots of performance drop).

## Module 4: Uncertainty Calibration Audit

Beyond accuracy, we evaluate **probability calibration** both in-domain and under cross-dataset shift. We apply standard post-hoc calibration methods (Temperature Scaling and Isotonic Regression) on a validation split of each training dataset, and then measure their effect on both held-out same-domain and cross-domain test sets. Literature shows that covariate shift **breaks calibration**: a model calibrated on source data can become miscalibrated on a shifted target [35†L223-L228]. Specifically, Guo *et al.* (2017) temperature scaling assumes i.i.d. data – when this is violated, the mapping no longer aligns predicted confidence with true probability. Our framework quantifies this *calibration degradation* using Expected Calibration Error (ECE) and Maximum Calibration Error (MCE) [37†L578-L584].

Formally, we bin predictions into  $M$  intervals and compute

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|, \quad \text{MCE} = \max_m |\text{acc}(B_m) - \text{conf}(B_m)|,$$

where  $\text{acc}(B_m)$  is the empirical accuracy in bin  $B_m$  and  $\text{conf}(B_m)$  is average predicted confidence [37†L578-L584]. We compute ECE/MCE for (a) the in-domain test split, and (b) the out-of-domain test split. We expect significantly higher ECE/MCE under cross-dataset evaluation, highlighting calibration failure.

We also assess the clinical risk of miscalibration. A key metric is the frequency of **overconfident errors**: for example, the average confidence among wrong predictions on the OOD set. Intuitively, a model outputting e.g. 99% confidence when incorrect is dangerous. We will report confusion matrices and reliability diagrams for both domains. As a concrete illustration, Cearn *et al.* (2020) show a case where an uncalibrated model gave a 10% predicted probability of response (implying 90% confidence of failure) but after calibration it gave 47% [48†L238-L247]. In that example the clinician changed course. In our context, we will discuss how a model that is *overconfident* on foreign-domain cough sounds could

mislead clinicians (e.g. false negative vs false positive predictions). Reporting calibration errors (ECE/MCE) alongside accuracy makes the clinical implications explicit: a highly confident but uncalibrated classifier has low **decision reliability**, which can be quantitatively assessed by these metrics [48†L238-L247] [37†L578-L584] .

## Module 5: Implementation Roadmap (PyTorch + HuggingFace)

We outline a step-by-step plan for coding this framework, suitable for a Codex-like implementation agent:

- **(a) Data Handling:** Implement a `torch.utils.data.Dataset` that yields triplets. Using the metadata tables, match each subject's Cough, Breath, and Speech recordings into one example. The `__getitem__` should load the raw audio files (e.g. via `torchaudio.load`) and return `(waveform_cough, waveform_breath, waveform_speech, label, confound_id)`. The `confound_id` encodes e.g. device or noise category. Ensure normalization: zero-mean unit-variance or standardized sampling rate (e.g. resample all to 16 kHz if needed). Use `DataLoader` for batching.
- **(b) SSL Backbone Hooks:** Use HuggingFace Transformers. For example, load a pretrained Wav2Vec2 model and processor:

```
from transformers import Wav2Vec2Processor, Wav2Vec2Model
processor = Wav2Vec2Processor.from_pretrained('facebook/wav2vec2-base')
model_w2v = Wav2Vec2Model.from_pretrained('facebook/wav2vec2-
base').eval().to(device)
```

For each input waveform, apply

`processor(waveform, sampling_rate, return_tensors='pt')` then `model_w2v(**inputs)` to get `.last_hidden_state` (shape `[T, D]`). These features are detached from the graph (no gradient). Do this for cough, breath, speech independently. Stack or keep them separate as needed.

- **(c) Cross-Attention Fusion Module:** Define PyTorch `nn.Module` for the fusion block. For example, one can use three multi-head attention layers (c->b&s, b->c&s, s->c&b) and appropriate projections. Pseudocode:

```
class CrossModalFusion(nn.Module):
    def __init__(self, dim, heads):
        super().__init__()
        self.attn_cb = nn.MultiheadAttention(dim, heads)
        self.attn_cs = nn.MultiheadAttention(dim, heads)
        self.attn_bc = nn.MultiheadAttention(dim, heads)
        # ... similarly for b->others, s->others
        self.linear = nn.Linear(dim*3, dim) # to combine modalities
    def forward(self, Hc, Hb, Hs):
        Qc = Hc; Qb = Hb; Qs = Hs
        # Cough attends to Breath
        A_cb, _ = self.attn_cb(Qc, Hb, Hb)
        # Cough attends to Speech
```

```

A_cs, _ = self.attn_cs(Qc, Hs, Hs)
# Combine cough's attended output
Hc_fused = A_cb + A_cs + Hc # residual connection
# Similarly compute Hb_fused, Hs_fused
# ...
# Optionally combine all modalities into one representation:
H_concat = torch.cat([Hc_fused.mean(0), Hb_fused.mean(0),
Hs_fused.mean(0)], dim=-1)
fused = self.linear(H_concat) # final fused feature vector
return fused

```

Normalize and apply dropout as usual. This block captures the query-key interactions described mathematically above [46†L88-L94] .

- **(d) Loss and Training:** After fusion, pass the fused vector through the classification head  $g_\phi$  (e.g. an MLP) to predict the respiratory label, and through an adversarial confound head  $d_\psi$  to predict the confound\_id. Compute `loss_clinical = CrossEntropyLoss(y_pred, y_true)` and `loss_confound = CrossEntropyLoss(c_pred, c_true)`. Insert a gradient reversal function before the confound head so that when backpropagating `loss_confound`, the encoder's weights are updated with inverted gradient. In code, one can implement GRL as a custom autograd Function or use libraries like `torchrevgrad`. The combined loss is `loss_total = loss_clinical -  $\lambda$  * loss_confound`. In each training step, do:

```

optimizer.zero_grad()
L = loss_clinical - lambda_val * loss_confound
L.backward()
optimizer.step()

```

This realizes the objective  $L_{total}$ . Ensure that only the confound head is updated to minimize `loss_confound`, while the encoder is updated to maximize it (via GRL).

- **(e) Validation and Logging:** As the model trains, log in-dataset and cross-dataset accuracy, as well as ECE/MCE. Use a held-out split of Coswara to tune temperature for calibration, and similarly for COUGHVID. After each epoch, evaluate on the opposite dataset without adaptation to monitor OOD performance. Save model checkpoints based on combined criteria (e.g. best cross-dataset AUC).
- **(f) Hyperparameters & Reproducibility:** Document all hyperparameters (learning rate, batch size,  $\lambda$  value, number of attention heads, etc.) and use fixed random seeds. Prepare scripts to fully reproduce experiments.

Each component above should be modular and extensively commented. This roadmap provides a clear blueprint: (a) a triplet Dataset class to fetch matched cough/breath/speech waveforms; (b) HuggingFace-backed feature extraction calls; (c) a PyTorch module implementing the cross-attention fusion as sketched; (d) the dual-head classifiers with GRL and losses; (e) the training loop integrating all parts. Together, they constitute a rigorously specified pipeline ready for implementation.

**Sources:** Established works on adversarial de-biasing and attention underpin this design [19†L143-L151] [20†L7-L15] [46†L88-L94] [37†L578-L584] [48†L238-L247] .

